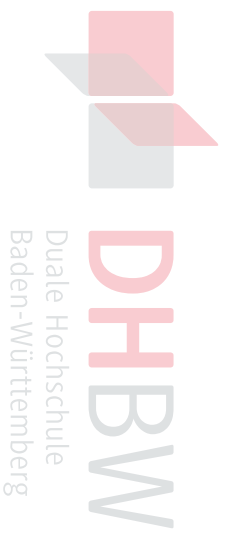


API DOKUMENTATION



Was ist eine API?

API DEFINITION

API = Application Programming Interface

- 🔌 Schnittstelle zwischen verschiedenen Softwarekomponenten
- 📋 Definiert, wie Programme miteinander kommunizieren
- 🎯 Legt fest: Welche Funktionen verfügbar sind
- 🔄 Bestimmt: Welche Daten eingegeben werden müssen
- 🔄 Spezifiziert: Welche Antworten zurückgegeben werden

API ARTEN



Web APIs:

Über das Internet zugänglich (z.B. REST, GraphQL)



Bibliotheks-APIs:

Teil von Softwarebibliotheken (z.B. Java Standard Library)



Betriebssystem-APIs:

Schnittstellen zu Betriebssystemfunktionen (z.B. Windows API)



Hardware-APIs:

Kommunikation mit Hardwarekomponenten (z.B. Drucker-APIs)



DHBW

Duale Hochschule
Baden-Württemberg

API DOKUMENTATION

Warum ist API Dokumentation wichtig?

- 📖 **Bedienungsanleitung:** Zeigt, wie die API verwendet wird
- ⚡ **Schneller Einstieg:** Entwickler können sofort produktiv werden
- 🐛 **Fehler vermeiden:** Korrekte Parameter und Datenformate
- 🔄 **Wartbarkeit:** Änderungen werden dokumentiert und kommuniziert
- 👥 **Teamarbeit:** Andere Entwickler verstehen die Schnittstelle
- 💰 **Geschäftswert:** Mehr Nutzer durch einfache Integration

API-DOC FÜR JAVA



Java Klassenbibliothek:

Extrem umfangreich (mehrere 1000 Klassen)



Schnittstelle dokumentiert:

API-Dokumentation für Anwendungsprogrammierung



JavaDoc Tool:

Generiert HTML-Seiten aus dem JDK



Online verfügbar:

[Oracle Java SE 21 API Dokumentation](#)

EIGENE API DOKUMENTATION

⭐ **Wichtiger Teil der Softwareentwicklung**

🕒 **Oft vernachlässigt:** Wird als zeitaufwändig empfunden

🔄 **Ziel:** Bessere Wiederverwendbarkeit eigener Komponenten

💡 Was dokumentiert werden sollte

📝 **Klassenname und Zweck** dokumentieren

🔧 **Methodenname und Funktionalität** beschreiben

📏 **Typ und Anzahl der Parameter** spezifizieren

⚡ **Wirkung und Laufzeitverhalten** erklären

BEST PRACTICES FÜR JAVADOC

- ✓ **Prägnant und klar:** Kurz und verständlich formulieren
- 🎯 **Zielgruppenorientiert:** Für Entwickler, die die API nutzen werden
- 🔍 **Vollständig:** Alle öffentlichen Klassen, Methoden und Felder dokumentieren
- 🔧 **Beispiele:** Code-Snippets zur Veranschaulichung der Nutzung
- ⚠️ **Warnungen:** Hinweise auf potenzielle Fallstricke
- 🔄 **Aktualität:** Dokumentation bei Code Änderungen anpassen
- ℹ️ **Frage:** Fragt euch selbst, was ihr wissen wollt bzw. müsst, um euren eigenen Code verwenden zu können.
- ⚠️ **Separate Dokumentation:** Fehleranfällig und schwer aktuell zu halten
- ✓ **JavaDoc-Kommentare:** Direkt im Quellcode → automatische Generierung

JAVADOC SYNTAX

Ein Dokumentationskommentar...

 **beginnt mit /** und endet mit */**

 **besteht aus HTML-Text mit speziellen Javadoc-Tags (beginnend mit @)**

 **muss vor der entsprechenden Deklaration stehen**
(Klasse, Variable, Konstruktor oder Methode)

Ein Dokumentationskommentar ist zweigeteilt in

1. **Beschreibung:** Text bis zum ersten @-Tag, wobei der erste Satz als Zusammenfassung interpretiert wird
2. **Tag-Liste:** Alle @-Tags mit beschreibendem Text

 **Vollständige Tag-Referenz:**

[Oracle Javadoc Tool Documentation](#)

TYPISCHE JAVADOC TAGS

Kommentar-Syntax	Beschreibung
<code>@see</code> Klassenname oder Methodennamen	Verweis auf eine andere Klasse/Methode
<code>@version</code> Versionstext	Versions-Informationen
<code>@author</code> Autortext	Autoren-Informationen
<code>@param</code> Parametername Parametertext	Beschreibung der Parameter
<code>@return</code> Rückgabebetext	Rückgabewert-Informationen
<code>@throws</code> Exception-Name Text	Beschreibung möglicher Ausnahmen/Fehler
<code>{@link Verweis}</code>	Im Text eingebauter Verweis (wie <code>@see</code>)

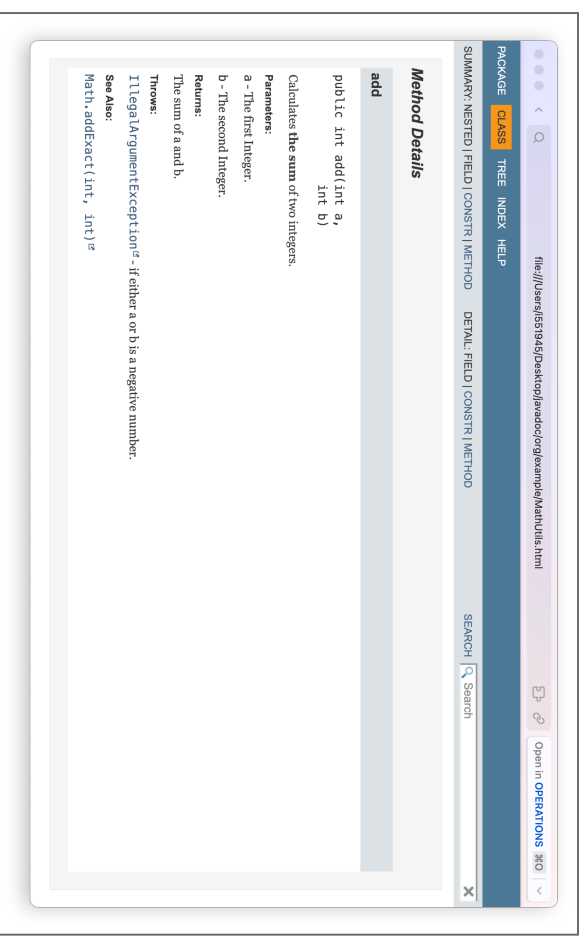
BEISPIEL JAVADOC



```
1  /**
2   * Calculates <b>the sum</b> of two integers.
3   *
4   * @param a The first Integer.
5   * @param b The second Integer.
6   * @return The sum of a and b.
7   * @throws IllegalArgumentException if either a or b is a negative number.
8   * @see Math#addExact(int, int)
9   */
10 public int add(int a, int b) {
11     if (a < 0 || b < 0) {
12         throw new IllegalArgumentException("Parameter must be non-negative");
13     }
14     return a + b;
15 }
```

BEISPIEL JAVADOC

```
1  /**
2   * Calculates <b>the sum</b> of two
3   * integers.<b>a</b>
4   *
5   * @param a The first Integer.
6   * @param b The second Integer.
7   * @return The sum of a and b.
8   * @throws IllegalArgumentException
9   *         if either a or b is a negative
10  *         number.
11  * @see Math#addExact(int, int)
12  */
13 public int add(int a, int b) {
14     if (a < 0 || b < 0) {
15         throw new
16             IllegalArgumentException("Parameter
17             must be non-negative");
18     }
19     return a + b;
20 }
```



JAVADOC GENERIEREN

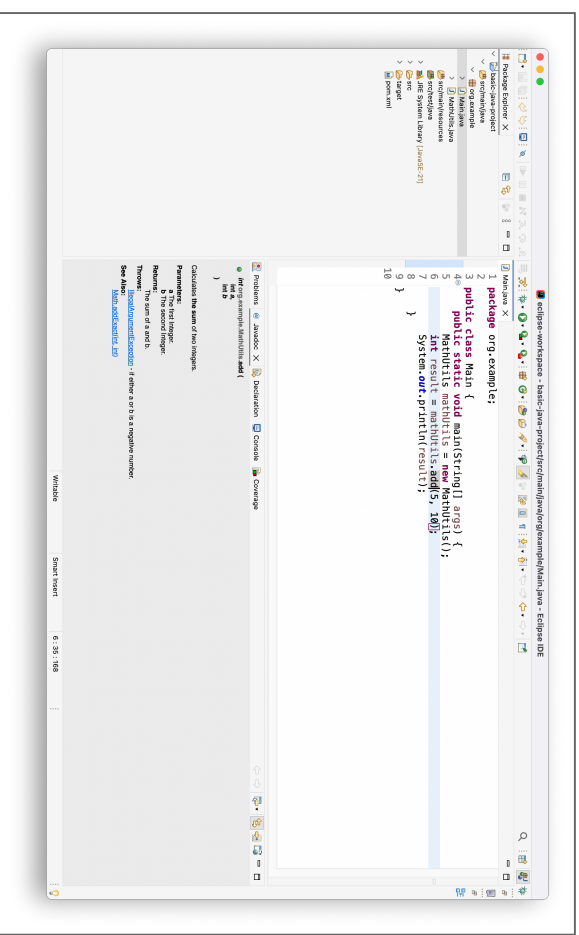


```
javadoc -d <target> Klasse.java
```

Parameter	Beschreibung
-author	@author wird mit in die Dokumentation der Klasse aufgenommen.
-version	@version wird mit in die Dokumentation der Klasse aufgenommen.
-private	Alle Klassen, Methoden und Variablen werden in die Dokumentation mit aufgenommen.
-protected	Nur protected und public Klassen, Methoden und Variablen werden in die Dokumentation mit aufgenommen. (Standardeinstellung)
-public	Nur public Klassen, Methoden und Variablen werden in die Dokumentation mit aufgenommen.

JAVADOC GENERIEREN

JAVADOC NUTZEN IN DER IDE



OPEN-API



 **OpenAPI Specification** (früher Swagger Specification)

 **Formaler Standard:** Definiert Spezifikationen zur Beschreibung von HTTP-APIs

 **Maschinen- und menschenlesbar:** Ermöglicht automatische Generierung von Dokumentation, Client-Bibliotheken und Server-Stubs

 **Werkzeuge:**

 **Swagger UI:** Interaktive API-Dokumentation

 **Swagger Codegen:** Generierung von Client- und Server-Code

 **Swagger Editor:** Online-Editor für OpenAPI-Spezifikationen